

1 HANDIN 1

QUESTIONS

Problems

Problem 1

There are three people (Alex, Ben, and Cody), one of whom is a knight, one a knave, and one a spy. The knight always tells the truth, the knave always lies, and the spy can either lie or tell the truth.

Alex says: "Cody is a knave." Ben says: "Alex is a knight." Cody says: "I am the spy."

Who is the knight, who is the knave, and who is the spy? Explain your logical reasoning.

Problem 2

Use truth tables to verify the following logical equivalence:

$$[p \rightarrow (q \vee r)] \leftrightarrow [(p \wedge \neg q) \rightarrow r].$$

Problem 3

Give the reasons for the steps verifying the following proof of $s \vee t$. Follow the approach shown in the book in Example 2.24 (page 73) and use the rules of Table 2.19, page 78. You can also use the equivalences on pages 58, 59, and 62.

Our premises are:

$$(a) \ p, \quad (b) \ p \rightarrow q, \quad (c) \ s \vee r, \quad (d) \ r \rightarrow \neg q.$$

We use the following steps:

- (a) p
- (b) $p \rightarrow q$
- (c) q
- (d) $r \rightarrow \neg q$
- (e) $q \rightarrow \neg r$
- (f) $\neg r$
- (g) $s \vee r$
- (h) s
- (i) $s \vee t$

Problem 4

For the universe of all integers, let $p(x)$, $q(x)$, and $r(x)$ be the following open statements:

$$p(x) : x > 0, \quad q(x) : x \text{ is even}, \quad r(x) : x \text{ is divisible by 7}.$$

Write the following statements in symbolic form.

- (a) At least one integer is even.
- (b) There exists a positive even integer.
- (c) No even integer is divisible by 7.
- (d) All positive integers are even.
- (e) There exists an even integer divisible by 7.

Find all statements that are false and provide a counterexample for each of those.

Problem 5

Using only the Peano definitions of addition and multiplication (see below), prove that for all natural numbers n it is true that

$$1 \cdot n = n.$$

Peano definitions:

$$\begin{aligned} 0 + a &= a & [+1], \\ s(x) + a &= s(x + a) & [+2], \\ 0 \cdot a &= 0 & [\cdot 1], \\ s(x) \cdot a &= (x \cdot a) + a & [\cdot 2], \\ 1 &= s(0) & [n.1]. \end{aligned}$$

Problem 6

Find the flaws in the following answer and proof, and fix them.

Question: For which natural numbers n is $n + n = n \cdot n$?

Answer: $n = 2$

Proof:

$$\begin{aligned} n + n &= n \cdot n && \text{given} \\ n \cdot 2 &= n \cdot n && \text{definition of multiplication} \\ 2 \cdot n &= n \cdot n && \text{commutativity of } \cdot \\ 2 &= n && \text{division by } n \\ n &= 2 && \text{commutativity of } = \end{aligned}$$

ANSWERS

2 Task 1

The strategy to solve this task is to analyze the logical consequences that might follow from the statements of the three characters in the problem. Although it is possible to solve the task by brute force, systematically testing every distribution of roles, the former approach was chosen to deduce each role. Since each role is unique to exactly one individual, we can use the exclusivity of roles together with the truth or falsity of each statement to rule out impossible cases. Following this reasoning shows that Alex must be the knight, Ben the spy, and Cody the knave.

First of all, Cody states that he is the spy. If Cody were the knight, then his statement “I am the spy” must be true. Cody cannot be two roles at once. Since the roles are mutually exclusive, this statement yields a contradiction. Cody cannot be the knight.

Consider Cody as a spy. His statement would be true, which is permissible for his role. Now consider Alex’s statement “Cody is a knave.” which would be false (since we established that Cody is a spy). Therefore, Alex cannot be the knight due to a false statement. Since the characters are mutually exclusive to the roles, eliminating both Alex and Cody means that Ben must be the knight. And if Ben is the knight, then his statement “Alex is a knight” must be true, which forces Alex to be the knight. This contradicts the previous deduction, which stated that Alex is not the knight. The contradiction implies that Cody is not the spy.

Since **Cody is neither a knight nor a spy, it makes a knave**. Then Alex’s statement “Cody is a knave” yields true.

Due to the previous deduction of Cody being the knave, Alex’s statement is true. This could imply both a knight or a spy, but if Alex were the spy, then Ben’s statement “Alex is a knight” would be false. This would force Ben to be the knave, but since the knave is already figured it only leaves a contradiction. This implies that **Alex must be the knight**, since the opposite is false. This also leaves **Ben as the spy**.

3 Task 2

The following truth table is used to prove that the two first-order expressions are a tautology.

p	q	r	$p \rightarrow (q \vee r)$	$(p \wedge \neg q) \rightarrow r$
0	0	0	1	1
0	0	1	1	1
0	1	1	1	1
1	0	1	1	1
0	1	0	1	1
1	1	0	1	1
1	0	0	0	0
1	1	1	1	1

Table 1: Truth table of $p \rightarrow (q \vee r) \Leftrightarrow (p \wedge \neg q) \rightarrow r$

4 Task 3

1) p (Premise)

2) $p \rightarrow q$ (Premise)

3) q by (1) and (2), Modus Ponens

$$\begin{array}{l} p \\ p \rightarrow q \\ \hline \therefore q \end{array}$$

4) $r \rightarrow \neg q$ (Premise)

5) $q \rightarrow \neg r$ by (4), Contrapositive

$$\begin{array}{l} r \rightarrow \neg q \\ \hline \therefore q \rightarrow \neg r \end{array}$$

6) $\neg r$ by (3), (5), Modus Ponens

$$\begin{array}{l} q \\ q \rightarrow \neg r \\ \hline \therefore \neg r \end{array}$$

7) $s \vee r$ (Premise)

8) s by (6), (7), Rule of Disjunctive Syllogism

$$\begin{array}{l} s \vee r \\ \neg r \\ \hline \therefore s \end{array}$$

9) $s \vee t$ by (8), Rule of Disjunctive Amplification

$$\begin{array}{l} s \\ \hline \therefore s \vee t \end{array}$$

5 Task 4

$$\mathbb{U} = \mathbb{Z}$$

a)

$$\exists x q(x)$$

b)

$$\exists x (p(x) \wedge q(x))$$

c)

$$\neg \exists (q(x) \wedge r(x))$$

d)

$$\forall x(p(x) \rightarrow q(x))$$

e)

$$\exists x(q(x) \wedge r(x))$$

c) is wrong, since $14/7 = 2$ and 14 is even. This is a counterexample.

d) is wrong, and a counterexample like 3 exists. It is a positive integer, but it is an odd number.

6 Task 5

1. $s(0) \cdot n$ by $[n.1]$
2. $(0 \cdot n) + n$ by $[.2]$
3. $(0) + n$ by $[.1]$
4. n by $[+.1]$

7 Task 6

The flaw in the proof arises at the step where both sides of the equation $2n = n \cdot n$ are divided by n . This implicitly assumes $n \neq 0$. Since 0 is often included in the natural numbers, it has to be explicitly noted that the division only holds for $n \neq 0$. The argument is therefore only correct for $n \in \mathbb{N} \setminus \{0\}$. For $n = 0$, the initial equation holds, but the step involving division is invalid without explicitly showing that it is not valid for all natural numbers.

$$n + n = n \cdot n \tag{1}$$

$$n \cdot 2 = n \cdot n \tag{2}$$

$$2 \cdot n = n \cdot n \tag{3}$$

$$2 = n, \quad n \neq 0 \tag{4}$$

$$2 = n \tag{5}$$

$$n = 2 \tag{6}$$

In this case, the exception $n = 0$ must be included in the answer.

$$n + n = 0 \iff n \cdot n = 0 \tag{7}$$

Therefore, the complete solution set is $n \in \{0, 2\}$.

Another possible solution is to factor the two terms and use the zero-product property theorem.

$$n \cdot 2 = n \cdot n \tag{1}$$

$$0 = n \cdot n - n \cdot 2 \tag{2}$$

$$0 = n(n - 2) \tag{3}$$

$$n(n - 2) = 0 \tag{4}$$

This clearly shows that $n \in \{0, 2\}$.

8 HANDIN 2

QUESTIONS

Problems

Problem 1

Using the well-ordering principle, check for each of the following sets whether they have a smallest element. Explain why.

- The set of all time points past noon.
- (*) The set of all powers of π : $\{\pi^i\}_{i \in \mathbb{N}}$.
- The set of all postcodes in Norway.

Problem 2

Prove the following equation by induction, where F_i is the Fibonacci sequence:

$$\sum_{i=1}^n F_i = F_{n+2} - 1.$$

Problem 3

Define the following two sequences recursively:

$$s_i = 42^{i+1}, \quad t_i = i \bmod 2.$$

Problem 4

Write 1000_{10} in base 7 and in base 12. Show all the important steps.

Problem 5

Prove the theorem below by induction using the Peano definitions of addition and multiplication.

$$\text{Th: } \forall n \in \mathbb{N} : n \cdot 0 = 0.$$

For your convenience, here are the definitions of addition and multiplication. The usual precedence of operations applies. Remember that you cannot use commutativity of $\cdot$ unless you prove it.

$$\begin{aligned} +1 : & \forall a \in \mathbb{N} : 0 + a = a \\ +2 : & \forall a \in \mathbb{N} : \forall x \in \mathbb{N} : s(x) + a = s(x + a) \\ \cdot 1 : & \forall a \in \mathbb{N} : 0 \cdot a = 0 \\ \cdot 2 : & \forall a \in \mathbb{N} : \forall x \in \mathbb{N} : s(x) \cdot a = x \cdot a + a \\ n.1 : & 1 = s(0) \end{aligned}$$

Problem 6

(*) The sequence s_i of natural numbers is defined recursively as follows:

$$s_0 = 1, \quad s_1 = 2, \quad s_{n+2} = s_n + 2.$$

Prove by induction for all indices i that

$$s_i < s_{i+1}.$$

ANSWERS

9 Task 1

Well-ordering principle

Statement: Every non-empty subset of the natural numbers $\mathbb{N}$ has a least smallest element **Well_ordering_principle**.

- **The set of all time points past noon.**

Treat noon as time 0. Then the times after noon form the interval $(0, \infty) \subset \mathbb{R}$. This set has no least element: for any $t > 0$, the smaller positive number $t/2$ is still in $(0, \infty)$. The set is continuous and uncountable. For any chosen time $t > 12 : 00$, there is always another time t' with $12 : 00 < t' < t$. Therefore, no smallest element exists. The well-ordering principle does not apply, because the set isn't a subset of $\mathbb{N}$.

- **All powers of π : $\{\pi^i\}_{i \in \mathbb{N}}$.**

Since $\pi > 1$, the numbers π^i get larger as i increases (if $i < j$ then $\pi^i < \pi^j$). So the smallest term is the one with the smallest index. The smallest element is 1.

- **The set of all postcodes in Norway.**

Postcodes in Norway range from 0001-9991 **Postal_code_search**. By the well-ordering principle, it has a least element. The smallest postcode is 0001.

10 Task 2

To prove the equation given using mathematical induction, the base case is as follows:

$$\sum_{i=1}^1 F_i = 1 \tag{5}$$

The induction hypothesis is accordingly:

$$\text{Assume } \sum_{i=1}^n F_i = F_{n+2} - 1 \tag{6}$$

Since F_{n+1} is the next term in the Fibonacci sequence, the induction step is:

$$\text{Prove } \sum_{i=1}^{n+1} F_i = \left(\sum_{i=1}^n F_i \right) + F_{n+1} \tag{7}$$

Henceforth, the induction hypothesis is substituted in the induction step.

$$(F_{n+2} - 1) + F_{n+1} \quad (8)$$

And by the Fibonacci recurrence:

$$F_{n+3} - 1 \quad (9)$$

11 Task 3

The task is solved by looking at the recurrence and seeing the pattern.

$$s(0) = 42$$

$$s(1) = 42^2 = 42 \cdot s(0)$$

$$s(2) = 42^3 = 42 \cdot s(1)$$

$$s(3) = 42^4 = 42 \cdot s(2)$$

The base case is:

$$s(0) = 42 \quad (1)$$

And the recursive step:

$$s(n+1) = 42 \cdot s(n) \quad , n \in \mathbb{N} \quad (2)$$

For $t = i\%2$:

$$t(0) = 0\%2 = 0$$

$$t(1) = 1\%2 = 1 = t(0) + 1$$

$$t(2) = 2\%2 = 0 = t(0) - 1$$

1. The base case is:

$$t(0) = 0 \quad (1)$$

2. Recursive step:

$$t(n+1) = t(n) + (-1)^n \quad (2)$$

12 Task 4

Convert 1000_{10} to base 7.

Find the largest integer q with $7q \leq 1000 < 7(q+1)$. Since $7 \cdot 142 = 994$ and $7 \cdot 143 = 1001$, we get $q = \lfloor 1000/7 \rfloor = 142$ and remainder $r = 1000 - 7 \cdot 142 = 6$. So $1000 = 7 \cdot 142 + 6$.

Then, doing repeated divisions by 7 to find the remainders:

$$1000 = 7 \cdot 142 + \mathbf{6}$$

$$142 = 7 \cdot 20 + \mathbf{2}$$

$$20 = 7 \cdot 2 + \mathbf{6}$$

$$2 = 7 \cdot 0 + \mathbf{2}$$

Reading the remainders from bottom to top which give us: 2626_7 .

Double checking: $2 \cdot 7^3 + 6 \cdot 7^2 + 2 \cdot 7 + 6 = 2 \cdot 343 + 6 \cdot 49 + 14 + 6 = 686 + 294 + 20 = 1000$.

Convert 1000_{10} **to base 12.** Digits in base 12 **Base_12_digits** are $0, 1, \dots, 9, A, B$ with $A = 10$, $B = 11$.

We need the largest integer q with $12q \leq 1000$. Check: $12 \cdot 83 = 996$ while $12 \cdot 84 = 1008 > 1000$. So $q = 83$ and the remainder is $r = 1000 - 12 \cdot 83 = 4$, $1000 = 12 \cdot 83 + 4$.

Then doing the same division as for the 7th base:

$$\begin{aligned} 1000 &= 12 \cdot 83 + 4 \\ 83 &= 12 \cdot 6 + 11 (= B) \\ 6 &= 12 \cdot 0 + 6 \end{aligned}$$

Reading then remainders from bottom to top gives $6B4_{12}$.

Double checking here as well: $6 \cdot 12^2 + B \cdot 12 + 4 = 6 \cdot 144 + 11 \cdot 12 + 4 = 864 + 132 + 4 = 1000$.

13 Task 5

First of all, proving the commutativity of $\cdot$ will make the task achievable. This is done through mathematical induction.

1. $P(0) = 0$
2. Assume $P(k) = k \cdot 0$
3. Prove $P(k+1) = s(k) \cdot 0$

Here it is implicit that $s(k)$ is the successor of k .

$$k \cdot 0 + 0 = 0 \text{ by } [\cdot.2] \tag{3}$$

And since we established that $k \cdot 0$, this is substituted in the former expression.

$$0 + 0 = 0 \tag{4}$$

$$0 = 0 \text{ by } [+.1] \tag{5}$$

14 Task 6

1. $s(0) = s_0 < s_1 = 1 < 2$
2. Assume $s(n) = s_n < s_{n+1}$
3. Prove $s(n+1) = s_{n+1} < s_{n+2}$

$$s_{n+(1+1)-1} = s_{(n+2)-1} = s_{n-1} + 2 \tag{6}$$

This can be used to express the following:

$$s_{n-1} + 2 < s_n + 2 \tag{7}$$

15 HANDIN 3

QUESTIONS

Problems

Problem 1

- (*) Write the following grammar in EBNF as concisely as possible. Use $|$ for alternatives, $\{ \}$ for repetitions, and $[]$ for optional parts.

$G = (\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, x, y, z, +, -, *, /, (,)\},$
 $\{\text{expression, expressionRest, term, termRest, factor, variable, constant, digit}\},$
 expression,
 $\{(\text{expression, term expressionRest}),$
 $(\text{expressionRest, } \varepsilon), (\text{expressionRest, expressionRest} + \text{term}), (\text{expressionRest, expressionRest} - \text{term}),$
 $(\text{term, factor termRest}),$
 $(\text{termRest, } \varepsilon), (\text{termRest, termRest} * \text{factor}), (\text{termRest, termRest} / \text{factor}),$
 $(\text{factor, constant}), (\text{factor, variable}), (\text{factor, (expression)}),$
 $(\text{variable, } x), (\text{variable, } y), (\text{variable, } z),$
 $(\text{constant, constant digit}), (\text{constant, digit}),$
 $(\text{digit, } 0), (\text{digit, } 1), (\text{digit, } 2), (\text{digit, } 3), (\text{digit, } 4), (\text{digit, } 5), (\text{digit, } 6), (\text{digit, } 7), (\text{digit, } 8), (\text{digit, } 9)\})$

Problem 2

- (*) Find a grammar for

$$a^n b^m c^o d^m e^n.$$

Problem 3

Construct a syntax tree with the grammar below for the string:

`if true then if false then true else false.`

$$s = \text{true} \mid \text{false} \mid \text{if } s \text{ then } s \mid \text{else } s$$

Problem 4

Find a derivation for the syntax tree below.



Figure 1: Caption

Problem 5

(*) Consider the following three derivations.

- (1) $E \Rightarrow L = R \Rightarrow L = 3 \Rightarrow L + L = 3 \Rightarrow 1 + L = 3 \Rightarrow 1 + 2 = 3$
- (2) $E \Rightarrow L = R \Rightarrow 1 = R \Rightarrow 1 = R + R \Rightarrow 1 = 2 + R \Rightarrow 1 = 2 + 3$
- (3) $E \Rightarrow L = R \Rightarrow L + L = R \Rightarrow 1 + L = R \Rightarrow 1 + 2 = R \Rightarrow 1 + 2 = 3$

Discuss the ambiguity of the grammar that allows these derivations. Give reasons.

Problem 6

Define an EBNF grammar that matches the example strings below. Use meaningful terminal and non-terminal names and make sure that the grammar can be used for other employees as well.

- A. Andreas Prinz, Professor, Department of ICT, Office A3-082, andreas.prinz@uia.no, +47 37 23 32 20
- B. Arne Wiklund, Universitetslektor, Department of ICT, Office A3-067, arne.wiklund@uia.no, +47 37 23 34 24 +47 984 47 348
- C. Sverre Lunøe-Nielsen, Førsteamanuensis, Department of Engineering Sciences, Office C5-085, sverre.lunoe-nielsen@uia.no, +47 37 23 37 46 +47 469 66 063

ANSWERS**16 Task 1**

The grammar can be written in EBNF as follows:

```

expression = term { (" + " | " - ") term }
term = factor { (" * " | "/" ) factor }
factor = constant | variable | "(" expression ")"
variable = "x" | "y" | "z"
constant = digit { digit }
digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

```

This grammar correctly defines arithmetic expressions with standard precedence: multiplication and division bind tighter than addition and subtraction.

17 Task 2

The group decided to present this in two ways: one where the superscript on c is 0 (i.e., no c 's), and one where it is treated as a free exponent (allowing any number of c 's).

With c and 0: Starting with $c^0 = x^0 = 1$, that means that there are no copies of c , and the "real" pattern is $a^n b^m d^m e^n$.

String starts with some number of a 's and ends with the same number of e 's. Inside the string, we have some number of b 's followed by the same number of d 's. So we now have two mirrored parts:

- a e
- b d

Grammar:

$$S \rightarrow a S e \mid T \quad T \rightarrow b T d \mid \varepsilon$$

$a S e$ means add one a in front and e in the end, and keep going with S inside. T is what happens in the middle when we stop adding a and e . The rule $b T d$ adds one b at the start and one d at the end; ε means stop (no more pairs).

The second way the group did this was by defining the exponent o as a non-terminal.

We will name it k to avoid confusion with the digit 0. The pattern is $a^n b^m c^k d^m e^n$.

We do the same as before, we wrap the string with matching a and e on the outside and matching b and d inside. In the middle, we allow any number of c 's (including none).

EBNF:

$$S \rightarrow a S e \mid T \quad T \rightarrow b T d \mid U \quad U \rightarrow c U \mid \varepsilon$$

U generates the block of c 's; ε means "stop".

18 Task 3

The grammar for this task is defined as

$$s \rightarrow \text{true} \mid \text{false} \mid \text{if } s \text{ then } s [\text{else } s]$$

From this definition, three production rules can be identified:

- Rule 1: $s \rightarrow \text{true}$
- Rule 2: $s \rightarrow \text{false}$
- Rule 3: $s \rightarrow \text{if } s \text{ then } s [\text{else } s]$

The string that needs to be checked is

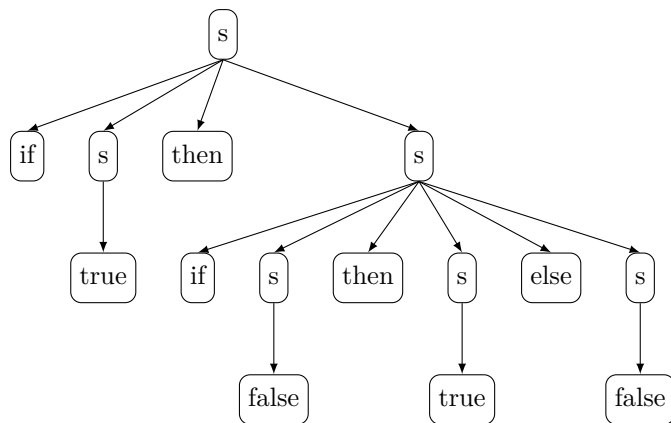
$$\text{if true then if false then true else false}$$

To show that the grammar can generate this string, both a derivation and a parse tree are included. A derivation tree explains how the start symbol is expanded step by step until only terminal symbols remain, and the leaves of the tree read from left to right form the final string **DerivationTree:online**. A parse tree represents the same process but focuses more on the sentence's structure. Parse trees are also useful when checking for ambiguity, since one string can sometimes be matched by more than one tree. **ParseTree:online**

The grammar rules are applied in the following order:

Step	Rule applied	Resulting string
1	Rule 3	if s then s
2	Rule 1	if true then s
3	Rule 3	if true then if s then s [else s]
4	Rule 2	if true then if false then s [else s]
5	Rule 1	if true then if false then true [else s]
6	Rule 2	if true then if false then true else false

The corresponding parse tree shows the same process in a hierarchical structure:

**Task 4**

In the given syntax tree, the nonterminal f expands to f^4 , where 4 is a terminal. Therefore, the correct derivation step is

$$f \Rightarrow f^4$$

and not $f \Rightarrow f^f$.

The derivation must follow the syntax tree exactly, expanding terminals and nonterminals according to the labels shown in the tree.

Task 5

The grammar is ambiguous.

The start symbol expands as

$$E \Rightarrow L = R$$

so every valid string must contain the symbol “=”.

Consider the string

$$1 + 2 + 3 = 4$$

This string can be generated in two different ways because of the rule

$$L \Rightarrow L + L$$

First parse:

$$(1 + 2) + 3 = 4$$

Second parse:

$$1 + (2 + 3) = 4$$

Since the same string has two different parse trees, the grammar is ambiguous.

19 Task 6

We define an EBNF grammar that matches the example strings.

GRAMMAR FOR NAME

<name>

consists of:

- First name
- Last name

<first-name>

First name is a required part of the name that consists of any sequence of letters including special characters from different alphabets.

<last-name>

Can consist of multiple strings of letters, optionally separated by whitespace or a hyphen. Example: "Lunøe-Nielsen".

<name-segment>

A name segment is a string of letters in the last name that is separated by a hyphen.

```
1 <name> = <first-name>, " ", <last-name>;
2 <first-name> = any sequence of letters
3 <last-name> = <name-segment> {"-", <name-segment>};
4 <name-segment> = any sequence of letters
```

GRAMMAR FOR TITLE

<title>

A title is an employee's job title. A title includes a string of letters, including special characters.

```
1 <title> = any sequence of letters, including special characters
```

GRAMMAR FOR DEPARTMENT

<department>

A department is a part of an organisation that employees belong/ work at.

<department-name>

The department name consists of sequences of any strings and letters from any alphabet from different languages. Department names can also include abbreviations.

```
1 <department> = "department of" " ", <department-name>
```

GRAMMAR FOR OFFICE

<office>

Office refers to a specific room or location for the employees. The office code is alphanumeric and is prefixed by "Office".

<office-code>

The office code is a unique identifier for all offices. it consists of both numbers and letters. Example: A3-082

```
1      <office> = "Office" , <office-code>;
2      <office-code> = alphanumeric sequence;
```

GRAMMAR FOR EMAIL <email>

Email is a unique digital address for an electronic mailbox that follows this format: username@domain.

<username>

The username is the first part of the email. It comes before the at sign (@). Usernames consist of letters, numbers, punctuation(.) and underscores (_).

Example: arne.wiklund

<domain>

The domain is the part of the email that comes after the 'at sign'. Domains often correspond to an organization or institution that provides the email service. An example of a domain name is uia.

```
1      <email> = <username>, "@", <domain>;
2      <username> = sequence of letter, numbers, or punctuation;
3      <domain> = domain name;
```

GRAMMAR FOR PHONE

<phone-number>

A phone number is a unique sequence of digits that starts with a plus (+) symbol and the country code. There can be multiple phone numbers listed in the strings which are separated by spaces.

“+”

Required at the beginning of a phone number before an international code.

<country-code>

A sequence of digits that represents the dialing code for the country of the employees. Example 47 (Norway's country code).

<digit-sequence>

A digit sequence comes after the country code. There can be multiple digit sequences separated by a space.

```
1      <phone-number> = "+", <country-code> , <digit-sequence>;
2      <country-code> = sequence of digits
3      <digit-sequence> = sequence of digits
```

PUTTING IT ALL TOGETHER

When we put this all together, we get:

```
<employee> = <name> , " , " , <department> , " , " , <office> , " , " , <email> , " , " ,
<phone-number> , { " " , <phone-number> };
```

where:

- Name: Andreas Prinz
- Title: Professor
- Department: Department of ICT
- Office: A3-082
- Email: andreas.prinz@uia.no
- Phone number: +47 37 23 32 20

20 HANDIN 4

QUESTIONS

Problems

Problem 1

Translate the following NFA to a DFA.

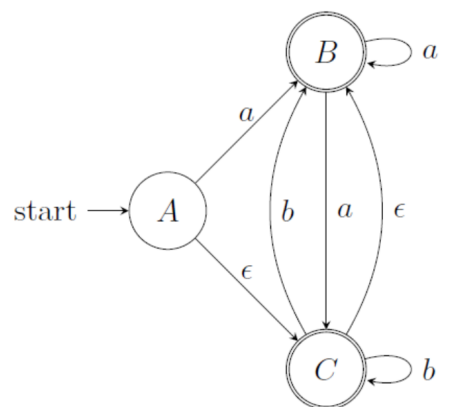


Figure 2: Caption

Problem 2

Translate the following regular expressions into one NFA:

- **keyword:** $0(12)^*$
- **special:** $121?$
- **number:** $[0123]^+$

Problem 3

Minimize the following DFA.

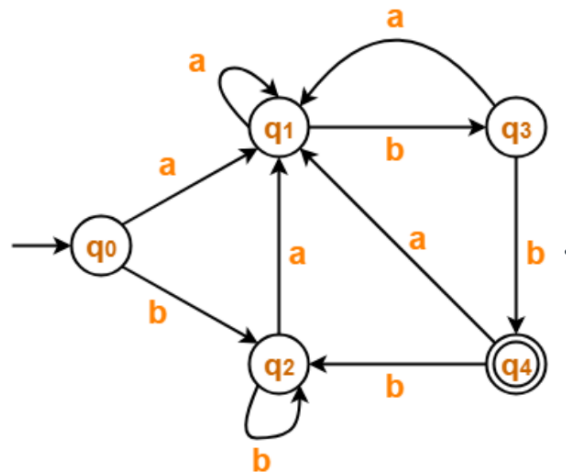


Figure 3: Caption

Problem 4

(*) When you translate a regular expression to an NFA, and then to a DFA, and then minimize it, do you get a unique result at the end?

Problem 5

Translate the following grammar to an NFA:

$$A \rightarrow aB \mid bA \mid b$$

$$B \rightarrow aC \mid bB$$

$$C \rightarrow aA \mid bC \mid a$$

Problem 6

Translate the regular expression

$$0^*11?0(01)^+$$

to a right-regular grammar.

Write the grammar as a 4-tuple

(Terminals, NonTerminals, startSymbol, Rules).

Remember that a right-regular grammar has only rules of the form

$$(A, t), (A, tB), (A, B), \text{ or } (A, \varepsilon),$$

where t is a terminal and A, B are nonterminals.

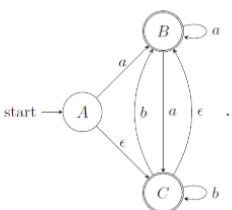
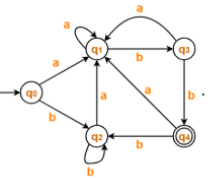
1. Translate the following NFA to a DFA: 
2. Translate the following regular expressions into one NFA: keyword: $0(12)^*$, special: $121?$, number: $[0123]^+$.
3. Minimize the following DFA: 
4. (*) When you translate a regular expression to an NFA, and then to a DFA, and then minimize it - do you get a unique result at the end?
5. Translate the following grammar to an NFA: $A \rightarrow aB \mid bA \mid b$, $B \rightarrow aC \mid bB$, $C \rightarrow aA \mid bC \mid a$
6. Translate the regular expression $0^*1[1?0](01)^+$ to a right-regular grammar. Write the grammar as a 4-tuple (Terminals, NonTerminals, startSymbol, Rules). B are nonterminals.

Figure 4: Enter Caption

ANSWERS

21 Task 1

21.1 NFA

$$S = \{A, B, C\}, \quad A = \{a, b\}, \quad s = A, \quad F = \{B, C\}.$$

Transitions (non-empty only):

$$T(A, a) = \{B\}, \quad T(A, \epsilon) = \{C\},$$

$$T(B, a) = \{B, C\},$$

$$T(C, b) = \{B, C\}, \quad T(C, \epsilon) = \{B\}.$$

All other $T(q, x) = \emptyset$.

21.2 How an NFA runs

Starts in A . At any time, it may take ϵ -moves (“free jumps”) without consuming input. When it reads a symbol (e.g. a or b), from its current state, it may follow *any* arrow with that symbol. A string is *accepted* if there exists some sequence of moves (including ϵ -moves wherever needed) that consumes the whole string and ends in an accepting state (B or C).

21.3 DFA

$\epsilon\text{-cl}(X)$ is used for the set of states reachable from X by taking zero or more ϵ -moves (free jumps that do not consume input). Since the NFA has ϵ -edges $A \xrightarrow{\epsilon} C$ and $C \xrightarrow{\epsilon} B$, we must use ϵ -closures in the subset construction.

21.3.1 ϵ -closures (needed for the construction)

$$\epsilon\text{-cl}(A) = \{A, B, C\}, \quad \epsilon\text{-cl}(B) = \{B\}, \quad \epsilon\text{-cl}(C) = \{B, C\}.$$

21.3.2 DFA states and start/accepting sets

Each DFA state is a subset of S .

$$S_0 = \epsilon\text{-cl}(\{A\}) = \{A, B, C\}, \quad S_1 = \{B, C\} \text{ (already } \epsilon\text{-closed).}$$

Accepting DFA states are those containing B or C :

$$F' = \{X \subseteq S \mid X \cap \{B, C\} \neq \emptyset\} = \{S_0, S_1\}.$$

DFA transition rule

For any DFA state $X \subseteq S$ and input $x \in \{a, b\}$:

$$\delta(X, x) = \epsilon\text{-cl}\left(\bigcup_{q \in X} T(q, x)\right).$$

Transitions computed

From $S_0 = \{A, B, C\}$:

$$\delta(S_0, a) = \epsilon\text{-cl}(T(A, a) \cup T(B, a) \cup T(C, a)) = \epsilon\text{-cl}(\{B\} \cup \{B, C\} \cup \emptyset) = \{B, C\} = S_1,$$

$$\delta(S_0, b) = \epsilon\text{-cl}(T(A, b) \cup T(B, b) \cup T(C, b)) = \epsilon\text{-cl}(\emptyset \cup \emptyset \cup \{B, C\}) = \{B, C\} = S_1.$$

From $S_1 = \{B, C\}$:

$$\delta(S_1, a) = \epsilon\text{-cl}(T(B, a) \cup T(C, a)) = \epsilon\text{-cl}(\{B, C\} \cup \emptyset) = \{B, C\} = S_1,$$

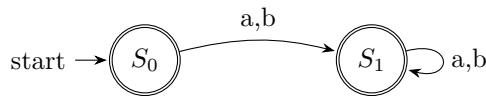
$$\delta(S_1, b) = \epsilon\text{-cl}(T(B, b) \cup T(C, b)) = \epsilon\text{-cl}(\emptyset \cup \{B, C\}) = \{B, C\} = S_1.$$

21.4 DFA summary

$$S' = \{S_0, S_1\}, \quad A' = \{a, b\}, \quad s' = S_0, \quad F' = \{S_0, S_1\}.$$

$$\delta(S_0, a) = S_1, \quad \delta(S_0, b) = S_1, \quad \delta(S_1, a) = S_1, \quad \delta(S_1, b) = S_1.$$

21.4.1 TikZ diagram of the DFA



22 Task 2

An NFA is constructed to recognize the following three regular expressions:

$$\text{keywords: } 0(12)^* \quad \text{special: } 121? \quad \text{number: } [0123]^+$$

The automaton is defined as the union of three smaller NFAs, combined through a global start state G . From G , ϵ -transitions lead into the start of each sub-NFA, allowing the machine to nondeterministically select one of the three branches. Accepting states are represented with double circles.

The transitions can be written as:

$$\textbf{Keywords: } K_0 \xrightarrow{0} K_1, \quad K_1 \xrightarrow{1} K_2, \quad K_2 \xrightarrow{2} K_1, \quad F_K = \{K_1\},$$

$$\textbf{Special: } S_0 \xrightarrow{1} S_1, \quad S_1 \xrightarrow{2} S_2, \quad S_2 \xrightarrow{1} S_3, \quad F_S = \{S_2, S_3\},$$

$$\textbf{Number: } N_0 \xrightarrow{0,1,2,3} N_1, \quad N_1 \xrightarrow{0,1,2,3} N_1, \quad F_N = \{N_1\}.$$

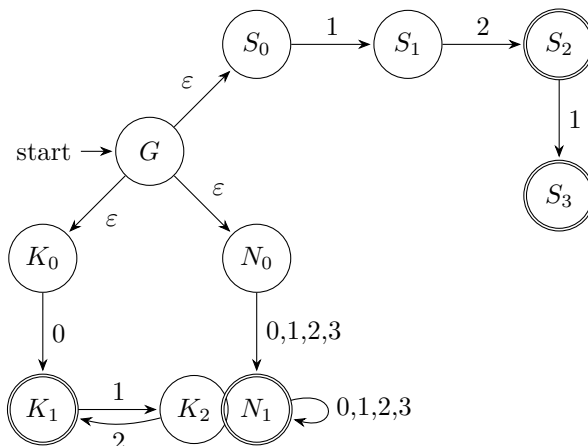
From the global start state:

$$G \xrightarrow{\epsilon} K_0, \quad G \xrightarrow{\epsilon} S_0, \quad G \xrightarrow{\epsilon} N_0.$$

The full set of accepting states is

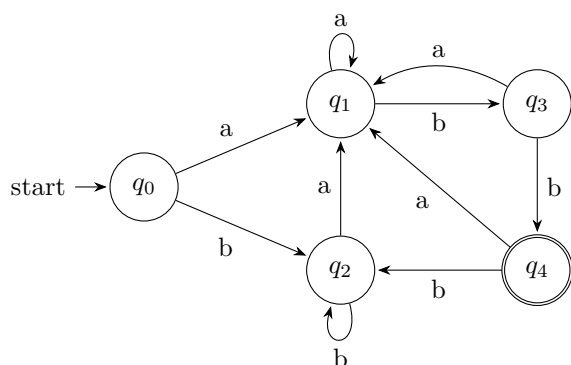
$$F = F_K \cup F_S \cup F_N = \{K_1, S_2, S_3, N_1\}.$$

The NFA begins in G and may immediately move into one of the three sub-NFAs by an ε -transition. In the *keywords* branch, state K_1 is accepting, producing strings such as 0, 012, and 01212. In the *special* branch, state S_2 accepts 12 and state S_3 accepts 121. In the *number* branch, state N_1 accepts any non-empty string over the alphabet $\{0, 1, 2, 3\}$, for instance 23 or 301.



23 Task 3

23.1 Starting DFA



23.2 Understanding DFA minimization

DFA minimization identifies and merges states that are *behaviourally equivalent*: two states are equivalent if, for every possible continuation of input, they either both lead to acceptance or both lead to rejection. To solve this the standard partition-refinement Method can be used to start separating the accepting and non-accepting states, and repeating until the sets no longer can split.

23.3 Minimization by partition refinement.

Setting up the Table with starting state of q_0 :

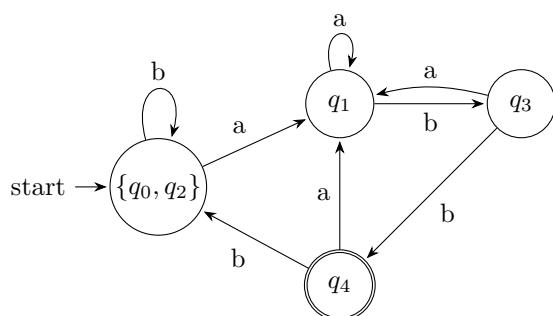
	a	b
q_0	q_1	q_2
q_1	q_1	q_3
q_2	q_1	q_2
q_3	q_1	q_4
q_4	q_1	q_2

For every state, read and check if which state both a and b leads to.
Each partition $P_0 - P_3$, uses its previous partition as starting point.

$$\begin{aligned} P_0 &= \{\{q_0, q_1, q_2, q_3\}, \{q_4\}\}, \\ P_1 &= \{\{q_0, q_1, q_2\}, \{q_3\}, \{q_4\}\}, \\ P_2 &= \{\{q_0, q_2\}, \{q_1\}, \{q_3\}, \{q_4\}\}, \\ P_3 &= P_2 \text{ (stable)}. \end{aligned}$$

Partion P_3 is stable since it does no longer split further, defining the new stats that shall be used in the minimized DFA $\{q_0, q_2\}, \{q_1\}, \{q_3\}, \{q_4\}$.

23.3.1 Minimized DFA



24 Task 4

Due to the properties of regular languages, a conversion from a regular expression to an NFA, DFA, and then a minimized DFA will always result in one unique minimal DFA. Below, the reasoning and deduction will be explained.

Firstly, converting a regular expression to a DFA can lead to different models with multiple possible transitions from the same state on the same input symbol (or via ϵ -transfers). Therefore, the NFA accepts any sequence of transitions that leads to acceptance. So, two different accepting paths can exist for the same input.

Any NFA can be converted into a DFA using subset constructions. This DFA is not necessarily minimal, and its number of states can vary depending on the initial NFA. After minimizing this DFA, however, the unique minimal DFA will always be an isomorph of the minimal DFA. In other words, the result is unique.

Below are two examples of NFAs that correspond to a regular expression that accepts bitstrings that end with 10 (e.g. 110, 10110, 1110). The regular expression for this is $(1|0)^*10$.

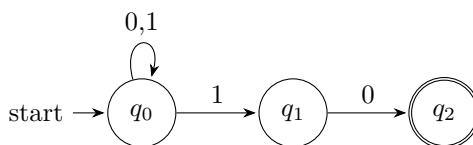


Figure 5: NFA of the regular expression $(1|0)^*10$
logiskemetoder-nfa

Another possibility for finding an NFA is to use Thompson's construction algorithm, which involves multiple ϵ -fractions. When converting the NFA to DFA as illustrated in Figure 6, we end up with the following automata:

When doing the same for the DFA created by Thompson's construction algorithm, the minimal DFA will always convert to the same one.

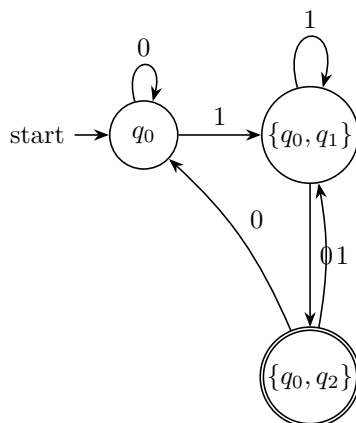


Figure 6: Minimal DFA for the language $(0|1)^*10$.

25 Task 5

Each nonterminal corresponds to an NFA state. Each production describes a transition. If the right-hand side ends with a nonterminal, we draw a transition to that state. If it ends with only a terminal, we draw a transition to a new accepting state.

For state A:

Transition a $\rightarrow$ goes to state B.

Transition b $\rightarrow$ loops back to self state A.

Transition b $\rightarrow$ goes to accepting state F.

For state B:

Transition a goes to state C.

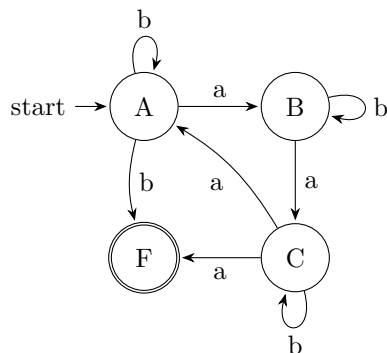
Transition b self loops to self state B.

For state C:

Transition a goes to state A.

Transition b loops back to self state C.

Transition a goes to accepting state F.



26 Task 6

By translating the regular expression $0^*1|1^*0|(01)^+$ into a 4-tuple grammar, the terminals, non-terminals, and productions have to be defined. The terminals are the literals which are 0 and 1.

$$T = \{0, 1\}$$

Continuously, the non-terminals are set to handle every operator that is in between the union operator ($|$). So for the expression 0^*1 , the non-terminal A is derived, B generates $1D^*0$, while D is symbolic for 0. The logic here is that $1D^*0$ generates the language $\{0, 10\}$, where $\epsilon \notin \{0, 10\}$. So even though the

regex branch of $1?$ is $(0|\epsilon)$, the language doesn't allow an empty string. Furthermore, C generates a 01 loop and E generates $(01)^+$. By this deduction, we get:

$$NT = \{S, A, B, C, D, E\}$$

Where the start symbol is S

Altogether, the 4-tuple will look like this:

$$(\{0, 1\}, \{S, A, B, C, D, E\}, S, \{(S, A|B|C), (A, 0A), (A, 1), (B, 1D), (B, 0), (C, 0E), (D, 0), (E, 1C), (E, 1)\})$$

27 HANDIN 5

QUESTIONS

Problems

Problem 1

Is the following code correct? Give reasons for your answer.

```
int bigger(int p1, int p2) { return (p1 > p2) ? p1 : p2; }
```

Problem 2

Why is testing not enough to ensure correctness?

Problem 3

For your programming language of choice, which possibilities do you have to define your own abstract data types?

Problem 4

Write an interface for an ADT `Boolean` with the functions `{True, False, And, Not}`. You only need to define the type and operations, not the semantics.

Problem 5

- (*) Extend the JML definition of Peano from the lecture with a function computing the minimum of two numbers.

Hint 1: This is Peano, and you cannot check something like $<$ yet.

Hint 2: You could define $\leq$ (`geq`) as follows:

$$\begin{aligned} \text{geq}(0, x) &= \text{true}, \\ \text{geq}(s(x), 0) &= \text{false}, \\ \text{geq}(s(x), s(y)) &= \text{geq}(x, y). \end{aligned}$$

Hint 3: You could define `min` recursively as follows:

$$\begin{aligned} \text{min}(0, x) &= 0, \\ \text{min}(x, 0) &= 0, \\ \text{min}(s(x), s(y)) &= s(\text{min}(x, y)). \end{aligned}$$

Problem 6

(**) Use refinement to provide a correct implementation of a function that solves a quadratic equation.

ANSWERS

28 Task 1

The function:

```
int bigger(int p1, int p2) {  
    return (p1 > p2) ? p1 : p2;  
}
```

Listing 1: bigger function declaration

is intended to return the larger of the two integer inputs. Intuitively, the expected behaviour can be stated as **if** $p1 \geq p2$ **then** $p1$ **else** $p2$.

The function itself lacks an explicit specification, and without one, correctness cannot formally be concluded. Program correctness is always relative to a specification, which defines the intended behaviour **specification-behaviour**.

If the name **bigger** is taken to define the intended behaviour, then the specification can be deduced: the function should return the maximum of its two arguments. The implementation uses the ternary operator $?:$, which is equivalent to an **if-else** construct. The condition $(p1 > p2)$ is evaluated: if true, $p1$ is returned; otherwise, $p2$ is returned. In the special case $p1 = p2$, the function returns $p2$, which is consistent with the specification since both values are equal.

Hence, under the assumed specification, the function correctly computes $\max(p1, p2)$. However, the absence of an explicit specification highlights a general risk: in less trivial programs, ambiguity about intended behaviour can lead to errors and misinterpretations in both development and verification.

29 Task 2

Testing can give confidence that a program works, but it is not enough to be sure that the program is correct. Testing only checks some chosen inputs, while correctness means that the program must give the right answer for every possible input. Since it is impossible to test all cases, testing alone cannot guarantee full correctness.

Consider the function:

```
int bigger(int p1, int p2) {  
    return (p1 > p2) ? p1 : p2;  
}
```

Listing 2: bigger function declaration

The goal of this function is to return the largest of the two numbers. By running tests with values like $(3, 5)$, $(-2, -7)$, or $(4, 4)$, the output seems correct. However, testing cannot try out all pairs of integers, so it cannot prove that the function always behaves as expected. Since the set of integers is infinite in principle (or finite in computers, but very large), it would take an infinite amount of tests to ensure correctness. Even if all the integers were tested, what would happen when you reach the integer limit bound? This could lead to failures even if the tests within the normal integer range would prove correctness. In principle, the function wants to represent the quantifier:

$$\forall p1, p2 \in \mathbb{Z}, bigger(p1, p2) = \max(p1, p2) \quad (8)$$

In reality, testing can only ever show:

$$bigger(p1, p2) = \max(p1, p2) \quad \text{for some finite set of pairs } (p1, p2) \quad (9)$$

To be certain, it is necessary to reason about the code. The function checks if $p1 > p2$. If this is true, it returns $p1$, which is the larger of the two numbers. Otherwise, it returns $p2$, which is then the larger number (or equal if both are the same). This logical argument demonstrates that the function always returns the correct result for any input, which testing alone cannot guarantee.

30 Task 3

In Haskell, to define new Abstract data types (ADTs), certain built-in keywords are used with different nuances. `haskell` data declarations can be used for sums and product types. `haskell` newtype is for zero-cost abstraction with strong typing. Below are examples of how they are used to define ADTs.

```
data Season = Spring | Summer | Autumn | Winter
```

Listing 3: Season data type definition

```
newtype Natural = MakeNatural Integer
```

Listing 4: Natural ADT definition

Beyond the core language features, tools such as Liquid Haskell extend Haskell with refinement types, which allow specifications and invariants to be attached directly to ADTs **liquid-haskell specification-invariant**. This is not a built-in facility, but demonstrates how Haskell's type system can be extended to enforce correctness properties.

31 Task 4

The interface for `Boolean` is defined as follows:

```
public interface Boolean {
    Boolean True();
    Boolean False();

    Boolean Not(Boolean b1); // Arity 1
    Boolean And(Boolean b1, Boolean b2); // Arity 2
}
```

Listing 5: Boolean interface

32 Task 5

```
public interface Peano {
    // @ requires p1 != null && p2 != null
    // @ ensures \result == (p1 == zero()) ? true : (p2 == zero())
    //               // ? false : geq(pred(p1), pred(p2))
    public boolean geq(Peano p1, Peano p2);

    // @ requires p1 != null && p2 != null
    // @ ensures \result == (geq(p1, p2) ? p2 : p1)
    public Peano min(Peano p1, Peano p2);
}
```

Listing 6: Peano interface with the `geq` and `min` expansion

The `geq` function is defined by recursion on the Peano structure

1. If `p1` is `zero`, then `geq(p1,p2)` is `true`.
2. If `p1` is a successor and `p2` is `zero`, then `geq(p1, p2)` is `false`
3. Otherwise, both are successors, and we reduce the problem to:

`java` `geq(succ(x), succ(y)) = geq(x,y).`

The `min` function is also recursive:

1. If either argument is `zero`, the minimum is `zero`.
2. If both are successors, then:

`java` `min(succ(x), succ(y)) = succ(min(x, y))`

These recursive calls reduce the inputs, so the recursion is well-founded. This means the problem gets strictly smaller for each recursive step. The recursion checks which Peano number hits the base case (0) first, and that is the smallest number.

33 Task 6

1. Initial specification

$[= (\text{InI}) \quad x : \in \{r \mid a * r * r + b * r + c = 0\}$

Where x should be assigned some root of the quadratic equation

- 2.

$[= (\text{Rew}) \quad \Delta := b * b - 4 * a * c; \quad x : \in \{r \mid a * r * r + b * r + c = 0\}$

3. Weaken to algebraic solution form

$[= (\text{WeA}) \Delta := b * b - 4 * a * c; \quad x : \in \{(-b + \text{sqrt}(\Delta))/(2 * a), (-b - \text{sqrt}(\Delta))/(2 * a)\}$

With the obligation:

$\forall r. (a * r * r + b * r + c = 0) \Leftrightarrow r \in \{\dots\}$

4. Case distinction on Δ

$[= (\text{InI}) \quad \Delta := b * b - 4 * a * c;$

$\{\Delta > 0\} x : \in \{(-b + \text{sqrt}(\Delta))/(2 * a), (-b - \text{sqrt}(\Delta))/(2 * a)\}$

$\{\Delta == 0\} x : \in \{-b/(2 * a)\}$

5. Rewrite nondeterministic choice

$[= (\text{Rew}) \Delta := b * b - 4 * a * c;$

$\{\Delta > 0\} \{x := (-b + \text{sqrt}(\Delta))/(2 * a) \sqcup (x := (-b - \text{sqrt}(\Delta))/(2 * a))\}$

6. Assignment

$[= (\text{Ass}) \quad \Delta := b * b - 4 * a * c;$
 $\{\Delta > 0\} r1 := (-b + \text{sqrt}(\Delta)) / (2 * a);$
 $r2 := (-b - \text{sqrt}(\Delta)) / (2 * a)$
 $\{\Delta == 0\} r1 := -b / (2 * a)$

34 HANDIN 6

QUESTIONS

Problems**Problem 1**

The relation $a \mid b$ on natural numbers (read “ a divides b ” or “ b is a multiple of a ”) is true if there is a natural number n such that

$$a \cdot n = b.$$

Check whether $\mid$ is reflexive, symmetric, antisymmetric, and transitive. Also determine whether it is an equivalence relation, a partial order, or a total order. Give reasons.

Hint: Be careful with 0.

Problem 2

Write the relation matrix for the Boolean **nor**. What is the neutral element for **nor**?

Problem 3

(*) Prove that the matrix order (slide 24 in the lecture) is a partial order. Is it a total order?

Problem 4

Let

$$P = \{\{a, b, d\}, \{c, f\}, \{e, g\}\}$$

be a partition. Write the relation matrix for the implied equivalence relation.

Problem 5

(**) Let R be a relation on a set A .

Antisymmetry of R is defined as follows:

$$\text{Def.AS: } R \text{ is antisymmetric if } \forall a, b \in A : (a, b) \in R \wedge (b, a) \in R \Rightarrow a = b.$$

A characterisation of antisymmetry is as follows:

$$\text{Ch.AS: } R \text{ is antisymmetric if } \forall a, b \in A : a \neq b \wedge (a, b) \in R \Rightarrow (b, a) \notin R.$$

Prove that Ch.AS is equivalent to Def.AS.

- (a) Prove that Ch.AS implies Def.AS.
- (b) Prove that Def.AS implies Ch.AS.

Problem 6

(*) What is the relation between the partitions of $\equiv_{10}$ and the partitions of $\equiv_{20}$?

ANSWERS

35 Task 1

Let $\mathbb{N} = \{0, 1, 2, \dots\}$ (include 0). Define

$$a \mid b \iff \exists n \in \mathbb{N} \text{ such that } a \cdot n = b.$$

Zero facts: For any a , $a \mid 0$ (pick $n = 0$). Also, $0 \mid b$ holds only when $b = 0$.

1. **Reflexive:** Means: $\forall a : a \mid a$. Every number divides itself since $a \cdot 1 = a$ (also true for $a = 0$).
Verdict: Yes.
2. **Symmetric:** Means: $\forall a, b : a \mid b \Rightarrow b \mid a$. If a divides b , it does not mean that b divides a . Example: $2 \mid 4$ but $4 \nmid 2$. **Verdict:** No.
3. **Antisymmetric:** Means: $\forall a, b : (a \mid b \wedge b \mid a) \Rightarrow a = b$. If $a \mid b$ and $b \mid a$, write $b = am$, $a = bn$. Then:
$$a = bn = (am)n = a(mn).$$
If $a = 0$ then $b = 0$. If $a > 0$, cancel a to get $mn = 1 \Rightarrow m = n = 1$, therefore $a = b$. **Verdict:** Yes.
4. **Transitive:** Means: $\forall a, b, c : (a \mid b \wedge b \mid c) \Rightarrow a \mid c$. If $b = am$ and $c = bn$, then $c = a(mn)$, so $a \mid c$. Zero case is consistent. **Verdict:** Yes.
5. **Equivalence relation:** Needs: reflexive + symmetric + transitive. Symmetry fails (e.g. $2 \mid 4$ but not $4 \mid 2$). **Verdict:** No.
6. **Partial order:** Needs: reflexive + antisymmetric + transitive. All hold, so $(\mathbb{N}, \mid)$ is partial order. **Verdict:** Yes.
7. **Total order:** Means: $\forall a, b$, either $a \mid b$ or $b \mid a$. Neither 2 divides 3 nor 3 divides 2.
Verdict: No.

36 Task 2

For $x, y \in \{0, 1\}$, the NOR operation is

$$x \downarrow y = \neg(x \vee y).$$

NOR evaluates to 1 only when both inputs are 0; otherwise it is 0.

$\downarrow$	0	1
0	1	0
1	0	0

Consider the binary relation $R \subseteq \{0, 1\} \times \{0, 1\}$ defined by:

$$(x, y) \in R \iff x \downarrow y = 1.$$

With the domain ordered as $(0, 1)$ for both rows and columns, the relation matrix $M = (M_{ij})$ is given by:

$$M_{ij} = \begin{cases} 1, & \text{if } i \downarrow j = 1, \\ 0, & \text{otherwise,} \end{cases} \quad \Rightarrow \quad M = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}.$$

(Only the pair $(0, 0)$ yields $\text{NOR} = 1$.)

A neutral element $e \in \{0, 1\}$ would satisfy $a \downarrow e = a$ and $e \downarrow a = a$ for all $a \in \{0, 1\}$. Using $a \downarrow e = \neg(a \vee e)$:

$$\begin{aligned} a = 0 : \quad & \neg(0 \vee e) = 0 \Rightarrow \neg e = 0 \Rightarrow e = 1, \\ a = 1 : \quad & \neg(1 \vee e) = 1 \Rightarrow \neg 1 = 1 \Rightarrow 0 = 1 \quad (\text{contradiction}). \end{aligned}$$

Hence no element e can satisfy the identity property for both inputs. Therefore, **NOR has no neutral element**.

37 Task 3

To prove a partial relation, its properties have to be proven. Partial relations are reflexive, antisymmetric and transitive. All three properties will be demonstrated below.

$$A \leq B \iff \forall i, j : A_{ij} \leq B_{ij}, \quad A, B \in \{0, 1\}^{m \times n} \quad (1)$$

1. Reflexive

For some set A :

$$\forall i, j (A_{ij} \leq A_{ij}) \quad (2)$$

This means that for any matrix A , with elements i and j , A has to be equal to itself since it has the same elements.

2. Antisymmetry

For some sets A and B :

$$(\forall i, j ((A_{ij} \leq B_{ij}) \wedge (B_{ij} \leq A_{ij})) \Rightarrow A_{ij} = B_{ij}) \quad (3)$$

Intuitively, given that A is less than or equal to B , and B is less than or equal to A , we can conclude that neither is actually “less than” ($<$). A and B are equal, in which the relation $\leq$ also covers.

3. Transitive

For some sets A , B and C :

$$\forall i, j ((A_{ij} \leq B_{ij}) \wedge (B_{ij} \leq C_{ij})) \Rightarrow A_{ij} \leq C_{ij} \quad (4)$$

The relation is not a total order, since there are incomparable matrices. The $\leq$ relation is total over the set of $\mathbb{R}$ and $\mathbb{Z}$ since all elements are comparable. With Boolean matrices, it is not possible to compare a Boolean matrix A that has 1 at positions where the Boolean matrix B does not have 1. This can be expressed as:

$$A \leq B \iff \{(i, j) | A_{ij} = 1\} \subseteq \{(i, j) | B_{ij} = 1\} \quad (5)$$

Since not all matrices are comparable even at the same domain $(n \times m)$, the relation is not total.

38 Task 4

Given $P = \{\{a, b, d\}, \{c, f\}, \{e, g\}\}$.

To start with the matrix dividing the partition into blocks, and working through each block by block following the $(x, y) \Leftrightarrow (y, x)$

For $\{a, b, d\}$: define 1 on $(a, a), (a, b), (a, d), (b, a), (b, b), (b, d), (d, a), (d, b), (d, d)$.

For $\{c, f\}$: define 1 on $(c, c), (c, f), (f, c), (f, f)$.

For $\{e, g\}$: define 1 on $(e, e), (e, g), (g, e), (g, g)$.

Remaining spaces are assigned 0, giving this matrix

	a	b	c	d	e	f	g
a	1	1	0	1	0	0	0
b	1	1	0	1	0	0	0
c	0	0	1	0	0	1	0
d	1	1	0	1	0	0	0
e	0	0	0	0	1	0	1
f	0	0	1	0	0	1	0
g	0	0	0	0	1	0	1

39 Task 5

Firstly, $\text{Def.AS} \Rightarrow \text{Ch.AS}$ is proven. Proof by contradiction is used below.

1. Assume Def.AS
2. Let $a, b \in A$ and assume the antecedent of $\text{Ch.AS}: a \neq b \wedge (a, b) \in R$
3. For contradiction, assume $(b, a) \in R$
4. From Def.AS and (2)+(3): $(a, b) \in R \wedge (b, a) \in R \Rightarrow a = b$
5. Contradiction with $a \neq b$. Therefore $(b, a) \notin R$
6. Since a, b were arbitrary, $\forall a, b((a \neq b \wedge (a, b) \in R) \Rightarrow (b, a) \notin R)$. Hence, Ch.AS

Next, $\text{Ch.AS} \Rightarrow \text{Def.AS}$ is proven.

1. Assume Ch.AS holds.
2. Let $a, b \in A$ and assume $(a, b) \in R \wedge (b, a) \in R$
3. Suppose $a \neq b$. Then by Ch.AS $a \neq b \wedge (a, b) \in R$ implies $(b, a) \notin R$. That contradicts $(b, a) \in R$ from (2).
4. So the supposition $a \neq b$ is impossible; hence $a = b$
5. Since a, b were arbitrary, $\forall a, b((a, b) \in R \wedge (b, a) \in R) \Rightarrow a = b$. That is Def.AS .

By definition, since $(\text{Def.AS} \Rightarrow \text{Ch.AS}) \wedge (\text{Ch.AS} \Rightarrow \text{Def.AS})$, that means the two first-order expressions are equivalent. Hence, $\text{Def.AS} \Leftrightarrow \text{Ch.AS}$.

40 Task 6

A congruence relation modulo n divides the set of integers $\mathbb{Z}$ into equivalence classes containing all integers that leave the same remainder when divided by n . The partition of $\mathbb{Z}$ modulo 10 consists of the ten classes $[0]_{10}, [1]_{10}, \dots, [9]_{10}$, whereas the partition modulo 20 consists of the twenty classes $[0]_{20}, [1]_{20}, \dots, [19]_{20}$.

Since every multiple of 20 is also a multiple of 10, the following implication holds:

$$x \equiv y \pmod{20} \Rightarrow x \equiv y \pmod{10}$$

Each equivalence class modulo 20 is contained within an equivalence class modulo 10. This shows that the partition included by $\equiv \pmod{20}$ is finer than the partition included by $\equiv \pmod{10}$.

Each congruence class modulo 10 can be expressed as the union of two congruence classes modulo 20 for $a = 0, 1, \dots, 9$:

$$[a]_{10} = [a]_{20} \cup [a + 10]_{20} \quad (6)$$

This occurs because numbers that differ by 10 belong to the same class modulo 10, but to distinct classes modulo 20.

41 HANDIN 7

QUESTIONS

Problem 7.1

Find the solution to the initial value problem given by

$$a_n + 0.5 a_{n-1} = 0, \quad \text{and} \quad a_0 = -1.$$

Problem 7.2

Assume that a_n satisfies the following conditions:

$$\begin{cases} a_n - 2a_{n-1} = n, \\ a_0 > 0. \end{cases}$$

- (a) Use induction to prove that $a_n > 0$ for all $n > 0$.
- (b) Prove that $a_n > a_{n-1}$ for all $n > 0$.

Problem 7.3

Let a_n be a solution to the initial value problem

$$\begin{cases} a_n = a_{n-1} + \frac{1}{2n}, \\ a_0 = 1. \end{cases}$$

Compute

$$\lim_{n \rightarrow \infty} a_n.$$

Problem 7.4

Solve the recurrence relation

$$a_{n+1} = \frac{(n+1)a_n + n}{n+2} \quad \text{for } n \geq 0,$$

where $a_0 = 1$.

Problem 7.5

Two players A and B play a game of chance. For each turn, a fair coin is flipped. If the result is heads, player A gives one euro to player B. Otherwise, player B gives one euro to player A. The game ends when one player is broke. The winner is the player with all the euros.

Assume that player A has 40 euros to start with, and player B has 60.

Let p_n be the probability that player A eventually wins, given that she currently has n euros.

If she has $n = 0$ euros, she has lost and the chance of winning is therefore zero. In other words, $p_0 = 0$. Likewise, $p_{100} = 1$, since $n = 100$ means that player A has all the euros and has therefore won.

Find a second-order homogeneous recurrence relation satisfied by p_n .

Problem 7.6

A bitstring is a word using only the letters 0 and 1. In this problem we will consider only those bitstrings which are of finite length k and do not contain two 1's next to each other.

For example, when $k = 1, 2$ and 3 , the following table lists all the allowed bitstrings:

$k = 1$	$k = 2$	$k = 3$
0	00	000
1	10	100
	01	010
		001
		101

Let a_k be the number of allowed words of length k . From the table we see that $a_1 = 2$, $a_2 = 3$, and $a_3 = 5$.

Prove that a_k satisfies the recurrence relation

$$a_k = a_{k-1} + a_{k-2} \quad \text{for all } k > 2.$$

Training Tasks

The following problems are not part of the handin problem set. However, if you want, you can submit solutions and get feedback.

Problem 7.7

Find the solution to the recurrence relation

$$a_n - a_{n-1} = 3n$$

such that $a_0 = 0$.

Problem 7.8

Solve the recurrence relation

$$a_n - 6a_{n-1} + 9a_{n-2} = 0 \quad \text{for } n \geq 2,$$

where $a_0 = 5$ and $a_1 = 12$.

ANSWERS**42 Problem 7.1**

Find a closed-form formula for the sequence defined by

$$a_n + \frac{1}{2}a_{n-1} = 0, \quad a_0 = -1.$$

The equation can be rewritten as:

$$a_n = -\frac{1}{2}a_{n-1}.$$

This sentence means: to get a new term, take the previous term and multiply it by $-\frac{1}{2}$. So every step:

- the sign flips (because the multiplier is negative),
- the size is cut in half (because the multiplier has size $\frac{1}{2}$).

Using $a_0 = -1$,

$$a_1 = -\frac{1}{2} \cdot a_0 = -\frac{1}{2} \cdot (-1) = \frac{1}{2},$$

$$a_2 = -\frac{1}{2} \cdot a_1 = -\frac{1}{2} \cdot \frac{1}{2} = -\frac{1}{4},$$

$$a_3 = -\frac{1}{2} \cdot a_2 = -\frac{1}{2} \cdot \left(-\frac{1}{4}\right) = \frac{1}{8}.$$

Now the pattern is clear: each time we multiply by $-\frac{1}{2}$. After n steps, we have applied the same multiplier n times.

Because we multiply by the same factor every time, this is a geometric sequence with ratio $-\frac{1}{2}$. Therefore,

$$a_n = \left(-\frac{1}{2}\right)^n \cdot a_0$$

Since $a_0 = -1$,

$$a_n = \left(-\frac{1}{2}\right)^n (-1) = -\left(-\frac{1}{2}\right)^n$$

Base case: For $n = 0$,

$$-\left(-\frac{1}{2}\right)^0 = -1$$

which matches a_0 .

The closed-form solution is:

$$a_n = -\left(-\frac{1}{2}\right)^n$$

It is a geometric sequence with ratio $-\frac{1}{2}$, which explains the alternating sign and the halving in size at each step.

43 Problem 7.2

a)

1. Base case ($n = 1$):

$$a_1 = 2a_0 + 1 > 0 \therefore a_0 > 0$$

2. Induction step:

For some $k \geq 1$, let $a_k > 0$. Then,

$$a_{k+1} = 2a_k + (k+1) > 2 \cdot 0 + (k+1) > 0$$

So by induction, $a_{k+1} > 0$ for all $n > 0$.

b)

Proving $a_n - 2a_{n-1} = n$, $a_0 > 0$:

We know that $a_n > 0$ for all $n > 0$

Base, using the recurrence with $n = 1$:

$$a_1 - 2a_0 = 1 \Rightarrow a_1 = a_0 + (a_0 + 1)$$

Hence:

$$a_1 - a_0 = a_0 + 1 > 0 \text{ Because } a_0 > 0, \text{ therefore } a_1 > a_0$$

For $n > 2$:

$$\text{Rewriting the recurrence as: } a_n - a_{n-1} = (a_n - 2a_{n-1}) + a_{n-1} = n + a_{n-1}$$

for $n \geq 2$ we have $a_{n-1} > 0$ by part a, so:

$$a_n - a_{n-1} = n + a_{n-1} > n \geq 2 > 0$$

Therefore $a_n > a_{n-1}$ for every $n \geq 2$ and we conclude that

$$a_n > a_{n-1} \text{ for all } n > 0$$

44 Problem 7.3

Computing the limit $\lim_{n \rightarrow \infty}$ requires turning the series into a sum. This is achieved by adding another term that follows the recurrence pattern.

$$a_n = a_{n-1} + \frac{1}{2^n} = a_{n-2} + \frac{1}{2^{n-1}} + \frac{1}{2^n} \quad (7)$$

Furthermore, the recurrence can be turned into a sum:

$$a_n = a_0 + \sum_{k=1}^n \frac{1}{2^k} \quad (8)$$

Now, this sum can be transformed into a geometric series following the equation:

$$S_n = a_1 \frac{1 - r^n}{1 - r} \quad (9)$$

And since the first term $(\frac{1}{2})^1 = \frac{1}{2}$, the following geometric series is:

$$\sum_{k=1}^n \frac{1}{2^k} = \frac{1}{2} \cdot \frac{1 - (\frac{1}{2})^n}{1 - \frac{1}{2}} = 1 - (\frac{1}{2})^n \quad (10)$$

And to compute $\lim_{n \rightarrow \infty}$:

$$\lim_{n \rightarrow \infty} a_n = \lim_{n \rightarrow \infty} 1 + (1 - (\frac{1}{2})^n) = \lim_{n \rightarrow \infty} 2 - (\frac{1}{2})^n \quad (11)$$

Since $\lim_{n \rightarrow \infty} (\frac{1}{2})^n$ converges to 0, the solution to the initial value problem is:

$$\lim_{n \rightarrow \infty} 2 - (\frac{1}{2})^n = 2 \quad (12)$$

45 Problem 7.4

$$a_{n+1} = \frac{(n+1)a_n + n}{n+2} \quad (1)$$

Multiplying this by $(n+2)$:

$$(n+2)a_{n+1} = (n+1)a_n + n \quad (2)$$

Now, let $b_n = (n+1)a_n$. This gives:

$$b_{n+1} - b_n = n \quad (3)$$

And since $b_0 = (0+1)a_0 = 1$, we can write it as a geometric series:

$$b_n = 1 + \sum_{k=0}^{n-1} k = 1 + \frac{n(n-1)}{2} \quad (4)$$

Now a_n can be expressed as:

$$a_n = \frac{b_n}{n+1} = \frac{n^2 - n + 2}{2(n+1)} \quad (5)$$

By partial fractions, the expression can be simplified:

$$a_n = \frac{n}{2} - 1 + \frac{2}{n+1} \quad (6)$$

And this is the solution.

46 Problem 7.5

Given the conditions where Head = A loses, Tails = A wins

Let p_n be the probability that player A eventually wins the game, given that A currently has n euros (so the total pot is 100 euros which is needed). The game ends when one of the players is broke.

If $n = 0$, A is already broke, thus $p_0 = 0$. If $n = 100$, A already has all the euros, thus $p_{100} = 1$.

From state n , where $1 \leq n \leq 99$, a coinflip happens giving the probabilities

- where A gain €1 and the next state is $n + 1$ the win probability from there is p_{n+1} ,
- where A lose €1 and the next state is $n - 1$ the win probability from there is p_{n-1} .

This gives

$$p_n = \frac{1}{2} p_{n+1} + \frac{1}{2} p_{n-1} \quad (7)$$

To rearrange (7), multiply it by 2 to remove fraction, and bring it to one side

$$2 * p_n = 2 * \frac{1}{2} p_{n+1} + 2 * \frac{1}{2} p_{n-1} \quad (8)$$

$$p_{n+1} - 2p_n + p_{n-1} = 0 \quad (9)$$

With the conditions given where $p_0 = 0$ and $p_{100} = 1$ this is the required 2nd-order homogeneous recurrence relation satisfied by (p_n) .

47 Problem 7.6

Let a_k denote the number of bitstrings of length k that do not contain two consecutive 1's. Given the initial values $a_1 = 2$, $a_2 = 3$, and $a_3 = 5$, prove that for all $k > 2$, the sequence satisfies the recurrence relation:

$$a_k = a_{k-1} + a_{k-2}$$

To prove the recurrence relation, we look at how valid bitstrings of length k can be constructed. Any bitstring that avoids two consecutive 1's must start in one of two ways:

- It starts with a 0
- It starts with a 10

These are the only possibilities, and no bitstring belongs to both groups.

Case 1: Starting with 0

If the bitstring begins with 0, then the remaining $k - 1$ positions can be filled with any valid bitstring of length $k - 1$. The number of such bitstrings is exactly a_{k-1} .

Case 2: Starting with 10

If the bitstring begins with 1, then to avoid two consecutive 1's, the next digit must be 0. So the string starts with 10, and the remaining $k - 2$ positions can be filled with any valid bitstring of length $k - 2$. The number of such bitstrings is exactly a_{k-2} .

Adding both cases gives the total number of valid bitstrings of length k :

$$a_k = a_{k-1} + a_{k-2}$$

This formula works for all $k > 2$ since we can always refer back to shorter valid bitstrings.

We know that:

$$a_1 = 2, \quad a_2 = 3, \quad a_3 = 5$$

Using our formula for $k = 3$:

$$a_3 = a_2 + a_1 = 3 + 2 = 5$$

This matches the given value of $a_3 = 5$.

The recurrence relation $a_k = a_{k-1} + a_{k-2}$ has been proven for all $k > 2$ by examining how valid bitstrings are built from shorter ones.

48 Problem 7.7

Let $d_n := a_n - a_{n-1}$. Then

$$d_n = a_n - a_{n-1} = (2a_{n-1} - 1) - a_{n-1} = a_{n-1} - 1 \quad (1)$$

Using $a_{n-1} = 2a_{n-2} - 1$ gives $a_{n-1} - 1 = 2(a_{n-2} - 1) = 2d_{n-1}$, hence

$$d_n = 2d_{n-1}, \quad d_2 = a_2 - a_1 = 2 \quad (2)$$

Therefore,

$$d_n = 2^{n-2} \cdot d_2 = 2^{n-2} \cdot 2 = 2^{n-1} \quad (3)$$

Now by telescoping, we get:

$$a_n - a_1 = \sum_{k=2}^n (a_k - a_{k-1}) = \sum_{k=2}^n 2^{k-1} = 2^n - 2. \quad (4)$$

With $a_1 = 3$, $a_n = 3 + (2^n - 2) = 2^n + 1$

49 Problem 7.8

IKKE GJORT

50 HANDIN 8

QUESTIONS

Problem 8.1

Prove that the symmetric group S_3 is not abelian.

Problem 8.2

Prove by brute force that $x \in \mathbb{Z}/6$ is a generator if and only if $\gcd(x, 6) = 1$.

Problem 8.3

Let S_4 be the symmetric group of all possible permutations of 4 symbols. Find a subgroup $H \subset S_4$ of order 4, and write down its elements.

Problem 8.4

Prove that the group $\mathbb{Z}/3 \times \mathbb{Z}/3$ is not cyclic.

Problem 8.5

Prove that if G is a cyclic group, then G is abelian.

Problem 8.6

Let $GL_n(\mathbb{R})$ be the set of all invertible $n \times n$ -matrices with real entries. It is a fact that matrix multiplication gives $GL_n(\mathbb{R})$ the structure of a group (you do not need to prove this in this problem).

Let $SO(n)$ be the subset of $GL_n(\mathbb{R})$ consisting of matrices of determinant equal to 1. Prove that $SO(n) \subset GL_n(\mathbb{R})$ is a subgroup.

Training Tasks

The following problems are not part of the handin problem set. However, if you want, you can submit solutions and get feedback.

Problem 8.7

Let G be a group of order 4. Write down all possible multiplication tables for G . Use this to prove that G must be abelian.

Problem 8.8

Find a generator $x = (x_1, x_2)$ for the group $\mathbb{Z}/6 \times \mathbb{Z}/5$.

Problem 8.9

Let $n > 1$ be a fixed natural number, and consider the n -fold direct product

$$\mathbb{Z}/2 \times \cdots \times \mathbb{Z}/2 = (\mathbb{Z}/2)^n.$$

An element $x \in (\mathbb{Z}/2)^n$ can be thought of as a string of bits of length n . For example, when $n = 4$, all elements of $(\mathbb{Z}/2)^4$ are

0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111,
1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111.

- (a) Compute the order of every element of $(\mathbb{Z}/2)^n$ for any n . (Try $n = 1, 2$ first, then generalize.)
- (b) Find all subgroups of $(\mathbb{Z}/2)^2 = \mathbb{Z}/2 \times \mathbb{Z}/2$.
- (c) Find all subgroups of $(\mathbb{Z}/2)^3 = \mathbb{Z}/2 \times \mathbb{Z}/2 \times \mathbb{Z}/2$.

Problem 8.10

Let n and q be natural numbers.

- (a) Let $c = \gcd(q, n)$, and assume that $c > 1$. Show that $d = \frac{n}{c}$ is a natural number such that $d < n$ and that $dq \equiv 0 \pmod{n}$.

- (b) Assume that there exists a natural number $d < n$ such that $dq \equiv 0 \pmod{n}$. Show that $\gcd(q, n) > 1$. (Hint: Assume that $\gcd(q, n) = 1$. Then show that this implies that $d \geq n$, which contradicts the assumption.)
- (c) Show that $\mathbb{Z}/n = \langle q \rangle$ if and only if $kq \not\equiv 0 \pmod{n}$ for all $1 \leq k < n$.
- (d) Combine (a)–(c) to show that $\mathbb{Z}/n = \langle q \rangle$ if and only if $\gcd(q, n) = 1$.

ANSWERS

51 Task 8.1

To determine whether S_3 is abelian, it suffices to find two elements that do not commute. Since S_3 is the closed group of all bijections of $\{1, 2, 3\}$ (i.e. permutations on the set), cycle notation will be used and the standard rightmost-first convention.

$$(12)(13) = (132), \quad (13)(12) = (123) \quad (5)$$

Since $(12)(13) \neq (13)(12)$, the elements (12) and (13) do not commute. Therefore, S_3 is a non-abelian group. ■

52 Task 8.2

Proving that $x \in \mathbb{Z}/6$ iff $\gcd(x, 6) = 1$ by brute force requires working in the group $(\mathbb{Z}_6, +)$ and checking every element by listing the cyclic subgroup.

$$\langle x \rangle = \{kx \pmod{6} : k = 0, 1, 2, 3, 4, 5\} \quad (6)$$

The list will be worked through from the smallest element $x = 0$ and upwards.

1. $x = 0$

$$\langle 0 \rangle \pmod{6} = \{0\} \quad (7)$$

This is not a generator for $\mathbb{Z}/6$, and $\gcd(0, 6) = 6$.

2. $x = 1$

$$\langle 1 \rangle \pmod{6} = \{0, 1, 2, 3, 4, 5\} \quad (8)$$

This is a generator for $\mathbb{Z}/6$, and $\gcd(1, 6) = 1$.

3. $x = 2$

$$\langle 2 \rangle \pmod{6} = \{0, 2, 4\} \quad (9)$$

This is not a generator for $\mathbb{Z}/6$, and $\gcd(2, 6) = 2$.

4. $x = 3$

$$\langle 3 \rangle \pmod{6} = \{0, 3\} \quad (10)$$

This is not a generator for $\mathbb{Z}/6$, and $\gcd(3, 6) = 3$.

5. $x = 4$

$$\langle 4 \rangle \pmod 6 = \{0, 4, 2\} \quad (11)$$

This is not a generator for $\mathbb{Z}/6$, and $\gcd(4, 6) = 2$

6. $x = 5$

$$\langle 5 \rangle \pmod 6 = \{0, 5, 4, 3, 2, 1\} \quad \blacksquare \quad (12)$$

This is a generator for $\mathbb{Z}/6$, and $\gcd(5, 6) = 1$.

It is now proven that x is only a generator of $\mathbb{Z}/6$.

53 Task 8.3

Let S_4 be the symmetric group on $\{1, 2, 3, 4\}$. This gives three possible ways to split the set: into these permutations.

$$(12)(34), (13)(24), (14)(23)$$

Each non-identity element is a product of two disjoint transpositions, example $(12)(34)$:

$$1 \mapsto 2, 2 \mapsto 1; 3 \mapsto 4, 4 \mapsto 3$$

$$1 \mapsto 1, 2 \mapsto 2, 3 \mapsto 3, 4 \mapsto 4 = e$$

$$(12)(34) \cdot (12)(34) = e, \quad (13)(24) \cdot (13)(24) = e, \quad (14)(23) \cdot (14)(23) = e,$$

Now every element's inverse lies in H , because they are their own inverse. Closure holds because the product of any two distinct non-identity elements is the third:

$$(12)(34) \cdot (13)(24) = (14)(23) \quad (13)$$

$$(12)(34) \cdot (14)(23) = (13)(24) \quad (14)$$

$$(13)(24) \cdot (14)(23) = (12)(34) \quad (15)$$

Now that H meets the closed operation criteria, by containing e , and the inverses $a^{-1} \in A$ for all $a \in A \subset G$ and isn't empty, it is a subgroup of S_4 of order 4.

$$H = \{(), (12)(34), (13)(24), (14)(23)\}$$

54 Task 8.4

Prove that the group $\mathbb{Z}/3 \times \mathbb{Z}/3$ is not cyclic.

The group $\mathbb{Z}/3 \times \mathbb{Z}/3$ consists of all ordered pairs (a, b) where $a, b \in \{0, 1, 2\}$, with addition performed modulo 3. The group therefore has $3 \times 3 = 9$ elements.

The order of an element (a, b) is the smallest positive integer n such that

$$n(a, b) = (0, 0).$$

This means $(na \bmod 3, nb \bmod 3) = (0, 0)$.

For any element (a, b) ,

$$3(a, b) = (3a \bmod 3, 3b \bmod 3) = (0, 0).$$

Thus, every element satisfies $3(a, b) = (0, 0)$, so the order of each nonzero element divides 3.

If $(a, b) = (0, 0)$, the order is 1. For any other element $(a, b) \neq (0, 0)$, the order is exactly 3, since smaller positive multiples do not give $(0, 0)$.

Therefore, every element in $\mathbb{Z}/3 \times \mathbb{Z}/3$ has order 1 or 3. No element has order 9, which means the group cannot be generated by a single element.

$$\mathbb{Z}/3 \times \mathbb{Z}/3 \text{ is not cyclic.} \quad \blacksquare$$

The group $\mathbb{Z}/3 \times \mathbb{Z}/3$ has exponent 3, while any cyclic group of order 9 must have exponent 9. This confirms that the group is not cyclic.

55 Task 8.5

A cyclic group is a group in which all elements can be generated from a single element. If G is cyclic, then there exists $g \in G$ such that every element of G can be written as a power of g :

$$G = \langle g \rangle = \{g^k \mid k \in \mathbb{Z}\} \quad (16)$$

Take arbitrary elements $a, b \in G$. Since G is cyclic, there exists integers m, n such that

$$a = g^m, \quad b = g^n \quad (17)$$

Using the laws of exponents in groups:

$$ab = g^m g^n = g^{m+n} \quad (18)$$

Similarly:

$$ba = g^n g^m = g^{m+n} \quad (19)$$

Since integer addition is commutative, $m + n = n + m$, therefore:

$$ab = g^{m+n} = g^{n+m} = ba \quad (20)$$

Hence $ab = ba$ for all $a, b \in G$, and G is abelian. $\blacksquare$

56 Task 8.6

To prove that a subset of a set GL_n is a subgroup, a set of properties needs to be proven. First of all, the subset $SO(n)$ cannot be empty, i.e., $SO(n) \neq \emptyset$. Secondly, the set needs to be closed with respect to the operation, in which case it is $\det(x)$. Then, the inverse of every element of $SO(n)$ is in $SO(n)$. Equation 1, 2, and 3 respectively prove all of the properties mentioned.

$$I_n \in GL_n(\mathbb{R}) \quad \text{and} \quad \det(I_n) = 1 \Rightarrow I_n \in SO(n). \quad (1)$$

$$\forall A, B \in SO(n) : \quad \det(AB) = \det(A) \det(B) = 1 \cdot 1 = 1 \Rightarrow AB \in SO(n). \quad (2)$$

$$\forall A \in SO(n) : \quad \det(A^{-1}) = (\det A)^{-1} = 1^{-1} = 1 \Rightarrow A^{-1} \in SO(n). \quad \blacksquare \quad (3)$$

57 HANDIN 9

QUESTIONS

Problem 9.1

Find natural numbers $x > 1$ and $y > 1$ such that

$$xy = 15.$$

Prove that your answer is correct.

Problem 9.2

Compute the order of the group P_n for $n = 3, 5, 15$.

Problem 9.3

Let $G = \mathbb{Z}/24$ and let H be the subgroup $\langle 8 \rangle \subset G$. Compute the coset $2 + H$.

Problem 9.4

Let $G = D_4$ be the dihedral group of symmetries of the square. Let $H = \langle r \rangle \subset G$ be the cyclic subgroup generated by the element r which rotates the square by an angle of $\pi/2$. Compute the index $[G : H]$.

Problem 9.5

Let $G = \mathbb{Z}/100$ and assume that $H \subset G$ is a subgroup. Prove that $|H| \neq 36$.

Problem 9.6

Let G be a finite group. Assume that $g \in G$ has order $|g| = 5$ and that $h \in G$ has order $|h| = 7$. Why must $|G| \geq 35$?

Training Tasks

The following problems are not part of the handin problem set. However, if you want, you can submit solutions and get feedback.

Problem 9.7

Use the extended Euclidean algorithm to find integers x and y such that

$$18x + 12y = \gcd(18, 12).$$

Problem 9.8

Let $G = \mathbb{Z}/12$ and let $H = \langle 4 \rangle \subset G$. Compute the index $[G : H]$.

Problem 9.9

Let $G = \mathbb{Z}/4 \times \mathbb{Z}/6$, let $g = (2, 2) \in G$ and let $H = \langle g \rangle$. Compute the index $[G : H]$.

ANSWERS**58 Task 9.1**

One example is $x = 8$ and $y = 2$. Indeed,

$$8 \cdot 2 = 16 \equiv 1 \pmod{15}.$$

59 Task 9.2

$$\begin{aligned}
 |P_3| &= |\{1, 2\}| = 2, \\
 |P_5| &= |\{1, 2, 3, 4\}| = 4, \\
 |P_{15}| &= |\{1, 2, 4, 7, 8, 11, 13, 14\}| = 8.
 \end{aligned}$$

The set P_{15} does not include multiples of 15 (i.e., $3 \cdot 5$). It can be written as

$$P_{15} = \{n \in \mathbb{Z} \mid \gcd(n, 15) = 1, 1 \leq n < 15\}.$$

60 Task 9.3

$$G = \mathbb{Z}/24 = \{0, 1, 2, 3, \dots, 23\}.$$

The subgroup generated by 8 is

$$\langle 8 \rangle = \{0, 8, 16\} \subset G.$$

Hence,

$$2 + H = \{2, 10, 18\}.$$

61 Task 9.4

Let

$$G = D_4 \quad \text{and} \quad H = \langle r \rangle = \{e, r, r^2, r^3\}.$$

The group D_4 has order 8 (four rotations and four reflections), while $|H| = 4$. Therefore,

$$[G : H] = \frac{|G|}{|H|} = \frac{8}{4} = 2.$$

62 Task 9.5

By Lagrange's theorem, the order of any subgroup H of a finite group G must divide the order of G . Thus, there exists an integer $k \in \mathbb{Z}$ such that

$$|G| = k \cdot |H|.$$

In this case,

$$k = \frac{|G|}{|H|} = \frac{100}{36}.$$

Since $\frac{100}{36} \notin \mathbb{Z}$, 36 does not divide 100. Hence,

$$|H| \neq 36.$$

63 Task 9.6

By Lagrange's theorem, the order of every element divides the order of the group. Since $|g| = 5$ and $|h| = 7$, both 5 and 7 divide $|G|$. The smallest positive integer divisible by both 5 and 7 is their least common multiple:

$$\text{lcm}(5, 7) = 35.$$

Therefore,

$$|G| \geq 35.$$

64 HANDIN 11**QUESTIONS**

Problem 11.1

Find the subset containing all elements of $(\mathbb{Z}/2)^5$ with weight less than or equal to 2:

$$\{ w \in (\mathbb{Z}/2)^5 \mid \text{wt}(w) \leq 2 \}.$$

Problem 11.2

A binary symmetric channel has a chance $p = 0.002$ of incorrectly transmitting a single bit. Let $w = 10100011$ be a word which is transmitted over the channel. Let $T(w)$ denote the received word.

What is the probability that $T(w) = w + e$ for some error string e with $\text{wt}(e) = 1$?

Problem 11.3

Let

$$E : (\mathbb{Z}/2)^2 \rightarrow (\mathbb{Z}/2)^5$$

be the block code given by

$$E(w_1 w_2) = w_1 w_2 w_3 w_4 w_5,$$

where $w_3 = w_1$, $w_4 = w_2$, and $w_5 = w_1 + w_2 \pmod{2}$.

(a) Find all possible code words

$$\{ E(w) \mid w \in (\mathbb{Z}/2)^2 \}.$$

(b) Find the largest integer k such that any error of weight less than or equal to k can be detected using this block code.

Problem 11.4

Given the generator matrix

$$G = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \end{pmatrix}$$

for the $(5, 2)$ block code given by

$$E(w) = w \cdot G \quad \text{for } w \in (\mathbb{Z}/2)^2,$$

(a) Compute all possible code words $E(w)$.

(b) Find the minimum Hamming distance between any two code words.

Training Tasks

The following problems are not part of the handin problem set. However, if you want, you can submit solutions and get feedback.

Problem 11.5

Given the binary symmetric channel with p and w as in Problem 11.2.

(a) What is the probability that $T(w) = 10100011$?

(b) What is the probability that $T(w) = 10101011$?

(c) What is the probability that $T(w) = w + e$ for some error string e with $\text{wt}(e) \leq 1$?

Problem 11.6

Let E be the block code of Problem 11.3. Find the largest integer k such that any error of weight less than or equal to k can be corrected using this block code.

Problem 11.7

Given the encoding function from Problem 11.4. Find the largest integer k such that any transmission error of weight less than or equal to k can be detected using this block code.

ANSWERS**Task 11.1**

To find the cardinality of the set containing all elements of $(\mathbb{Z}/2)^5$ with Hamming weight less than or equal to 2, we count the number of binary strings of length 5 with $k = 0, 1, 2$ ones.

The number of binary strings of length n with exactly k ones is

$$\binom{n}{k}.$$

Thus,

$$\binom{5}{0} + \binom{5}{1} + \binom{5}{2} = 1 + 5 + 10 = 16.$$

Hence, the set has cardinality 16, and the elements are

$\{00000, 00001, 00010, 00100, 01000, 10000, 00011, 00101, 01001, 10001, 00110, 01010, 10010, 01100, 10100, 11000\}.$

Task 11.2

The probability that exactly one transmission error occurs is given by

$$\binom{8}{1}p(1-p)^7,$$

where $p = 0.002$.

Thus,

$$\binom{8}{1} \cdot 0.002 \cdot (0.998)^7 \approx 0.01578,$$

which is approximately 1.58%.

Task 11.3

(a) The set of all codewords is

$\{00000, 01011, 10101, 11110\}.$

(b) To find the largest integer k such that any error of weight less than or equal to k can be *detected*, we compute the minimum Hamming distance $d_{\min}$ between distinct codewords.

	00000	01011	10101	11110
00000	0	3	3	4
01011	3	0	4	3
10101	3	4	0	3
11110	4	3	3	0

From the table, we see that

$$d_{\min} = 3.$$

A block code can detect all error patterns of weight

$$k \leq d_{\min} - 1.$$

Therefore, the largest such integer is

$$k = 2.$$

Task 11.4

(a) Since $w \in (\mathbb{Z}/2)^2$, there are $2^2 = 4$ possible inputs:

$$\{00, 10, 01, 11\}.$$

The corresponding codewords are

$$\begin{aligned} E(00) &= 00000, \\ E(10) &= 10110, \\ E(01) &= 01011, \\ E(11) &= 10110 \oplus 01011 = 11101. \end{aligned}$$

(b) We compute the Hamming distances between the codewords:

	00000	01011	10101	11101
00000	0	3	3	4
01011	3	0	4	3
10101	3	4	0	3
11101	4	3	3	0

Thus, the minimum Hamming distance is

$$d_{\min} = 3.$$